

1. Hard Link vs. Soft Link

Part A - Create the Links

Create a directory named link_lab and navigate to it.

```
$ mkdir link_lab
$ cd link_lab
$ pwd
/home/user/link_lab
```

Create a file named security.txt containing the following text:

Linux File System Security Lab

Confidential Security Report

```
$ cat > security.txt
Linux File System Security Lab
Confidential Security Report

$ cat security.txt
Linux File System Security Lab
Confidential Security Report
```

Create a hard link named security_hard.txt:

```
$ ln security.txt security_hard.txt
```

Create a symbolic link named security_soft.txt:

```
$ ln -s security.txt security_soft.txt
```

List all three files using a command that displays their inode numbers:

```
$ ls -li
total 8
1710576 -rw-rw-r-- 2 user user 62 Sep  6 13:46 security_hard.txt
1710577 lrwxrwxrwx 1 user user 12 Sep  6 13:46 security_soft.txt -> security.txt
1710576 -rw-rw-r-- 2 user user 62 Sep  6 13:46 security.txt
```

Record the inode numbers of:

- security.txt: 1710576
- security_hard.txt: 1710576
- security_soft.txt: 1710577

What difference do you observe?

security.txt and security_hard.txt share identical inode numbers because a hard link references the exact same inode table entry. In contrast, security_soft.txt possesses a distinct inode number because a symbolic link is an independent pointer file containing the target path string.

Part B - Examine File Metadata

Use `ls -l` to examine the three files. Identify the file type of each file from the first character of the permission field.

```
$ ls -l
total 8
-rw-rw-r-- 2 user user 62 Sep  6 13:46 security_hard.txt
lrwxrwxrwx 1 user user 12 Sep  6 13:46 security_soft.txt -> security.txt
-rw-rw-r-- 2 user user 62 Sep  6 13:46 security.txt
```

File Types Identified:

- security.txt: Regular file (-)
- security_hard.txt: Regular file (-)
- security_soft.txt: Symbolic link (l)

Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

- security.txt: link count 2
- security_hard.txt: link count 2
- security_soft.txt: link count 1

Explanation: Both security.txt and security_hard.txt are active directory references pointing to inode 1710576, giving it a total link count of 2. The symbolic link occupies its own distinct inode, which maintains a separate link count of 1.

Use the `stat` command on all three files and record the information:

```
$ stat security.txt
File: security.txt
Size: 62 Blocks: 8 IO Block: 4096 regular file
Device: 8,1 Inode: 1710576 Links: 2
Access: (0664/-rw-rw-r--) Uid: (1000/user) Gid: (1000/user)

$ stat security_hard.txt
File: security_hard.txt
Size: 62 Blocks: 8 IO Block: 4096 regular file
Device: 8,1 Inode: 1710576 Links: 2
Access: (0664/-rw-rw-r--) Uid: (1000/user) Gid: (1000/user)

$ stat security_soft.txt
File: security_soft.txt -> security.txt
Size: 12 Blocks: 0 IO Block: 4096 symbolic link
Device: 8,1 Inode: 1710577 Links: 1
Access: (0777/lrwxrwxrwx) Uid: (1000/user) Gid: (1000/user)
```

Parameter	Original	Hard Link	Soft Link
-----------	----------	-----------	-----------

Inode number	1710576	1710576	1710577
File type	Regular file (-)	Regular file (-)	Symbolic link (l)
Size	62 bytes	62 bytes	12 bytes
Link count	2	2	1
Permissions	-rw-rw-r--	-rw-rw-r--	lrwxrwxrwx

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

The original file and hard link store identical file sizes because both reference the exact same data blocks on disk. The symbolic link's size reflects only the character count of the target filename string (security.txt, which is 12 characters long).

Part C - Investigate readlink

Execute readlink on security_soft.txt. What information is displayed?

```
$ readlink security_soft.txt
security.txt
```

readlink outputs the exact relative target path string stored inside the symbolic link file.

Execute readlink on security_hard.txt. What do you observe?

```
$ readlink security_hard.txt
```

No output is returned.

Based on your observations, explain why readlink works differently for hard links and symbolic links.

A symbolic link explicitly holds a text path pointing to another file, allowing readlink to display that path. A hard link is a direct entry inside the directory structure referencing an inode, so it contains no stored path string to extract.

Use readlink -f security_soft.txt:

```
$ readlink -f security_soft.txt
/home/user/link_lab/security.txt
```

What does the -f option reveal?

The -f flag recursively evaluates all symbolic links to reveal the complete canonical absolute path of the ultimate destination file.

Part D - Content Modification

Append the following line to security.txt:

Unauthorized login detected

```
$ echo "Unauthorized login detected" >> security.txt
```

Display the contents using all three filenames:

```
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
```

Is the newly added content visible through both links? Explain why.

Yes. Modifying security.txt updates the underlying data blocks linked to inode 1710576. The hard link reads directly from this inode, while the soft link resolves security.txt to read the same inode.

Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?

```
$ echo "Security investigation started" >> security_hard.txt

$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started

$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started
```

Updates applied via security_hard.txt immediately propagate to security.txt and security_soft.txt because all operations target the identical physical file system blocks.

Part E - Rename Experiment

Rename security.txt to incident.txt:

```
$ mv security.txt incident.txt
$ ls -li
total 8
1710576 -rw-rw-r-- 2 user user 92 Sep  6 14:29 incident.txt
1710576 -rw-rw-r-- 2 user user 92 Sep  6 14:29 security_hard.txt
1710577 lrwxrwxrwx 1 user user 12 Sep  6 13:46 security_soft.txt -> security.txt
```

Try accessing the content through security_hard.txt. Does it still work?

```
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started
```

Yes, access remains unaffected because the hard link directly binds to inode 1710576 regardless of directory name changes.

Try accessing the content through security_soft.txt. Does it still work?

```
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

No. The symbolic link attempts to open security.txt, which no longer exists under that name.

Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?

```
$ readlink security_soft.txt
security.txt
```

The link retains the text string security.txt.

Explain why renaming the original file affects the symbolic link differently from the hard link.

Renaming alters a directory's text listing without modifying the inode number. Hard links point to inode numbers, so they stay functional. Soft links rely on filename resolution; once the targeted filename changes, the link fails to resolve.

Part F - Deletion Experiment

Recreate the links if necessary and then perform the following:

```
$ rm security_soft.txt
$ ln -s incident.txt security_soft.txt
$ ls -li
```

Delete the original file:

```
$ rm incident.txt
$ ls -li
total 4
1710576 -rw-rw-r-- 1 user user 92 Sep  6 14:29 security_hard.txt
1710577 lrwxrwxrwx 1 user user 12 Sep  6 14:39 security_soft.txt -> incident.txt
```

Check whether security_hard.txt can still be accessed:

```
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started
```

Yes, data access remains fully functional.

Check whether security_soft.txt can still be accessed:

```
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

No, file access fails.

Use ls -l and readlink to investigate the symbolic link after deleting its target:

```
$ ls -l
total 4
-rw-rw-r-- 1 user user 92 Sep  6 14:29 security_hard.txt
lrwxrwxrwx 1 user user 12 Sep  6 14:39 security_soft.txt -> incident.txt

$ readlink security_soft.txt
incident.txt
```

What is a dangling/broken symbolic link?

A dangling or broken symbolic link is a pointer file whose targeted pathname no longer exists on the filesystem.

Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode:

No. Deleting incident.txt reduced inode 1710576's link count from 2 to 1. The Linux kernel retains disk storage allocation until an inode's link count drops to zero and no open file descriptors remain.

Part G - Directory and Filesystem Behaviour

Create a directory named evidence. Attempt to create a hard link to this directory. Record the result:

```
$ mkdir evidence
$ ln evidence evidence_hard
ln: evidence: hard link not allowed for directory
```

The operation is rejected by the system kernel.

Create a symbolic link named `evidence_link` pointing to the evidence directory. Was the operation successful?

```
$ ln -s evidence evidence_link
$ ls -l
drwxrwxr-x 2 user user 4096 Sep  6 14:58 evidence
lrwxrwxrwx 1 user user   8 Sep  6 15:00 evidence_link -> evidence
```

Yes, symbolic links support directory targets.

Based on your experiment, compare hard and symbolic links with respect to directory linking:

Hard linking directories is prohibited to prevent infinite filesystem loops and broken hierarchy trees. Soft links can safely point to directories because they are handled as path redirections.

If multiple filesystems/mount points are available in the laboratory environment, attempt to create:

```
$ df -T
Filesystem      Type      1K-blocks    Used Available Use% Mounted on
/dev/sda1       ext4      38818336 18312668  18501600  50% /
Kali            vboxsf    211430396 200754492  10675904  95% /media/sf_Kali

$ echo "Cross filesystem test" > cross_test.txt
$ ln cross_test.txt /media/sf_Kali/cross_hard.txt
ln: failed to create hard link '/media/sf_Kali/cross_hard.txt': Invalid cross-device link

$ ln -s /home/user/Desktop/cross_test.txt /media/sf_Kali/cross_soft.txt
ln: failed to create symbolic link '/media/sf_Kali/cross_soft.txt': Operation not permitted
```

Creating a hard link across mount boundaries fails with Invalid cross-device link because inode numbers are unique only within a single filesystem. The soft link attempt failed with Operation not permitted due to mount permissions on the target system (vboxsf).

Final Comparison Table

Parameter	Hard Link	Symbolic Link
Inode number	Identical to original file	Unique, separate inode
File type	Regular file (-)	Symbolic link (l)

Link count	Increments original inode count	Independent count (typically 1)
File size	Identical to original file	Size of target path string
Stores target pathname?	No	Yes
readlink output	Returns empty output	Displays target path string
Effect of modifying target	Content changes reflect instantly	Content changes reflect instantly
Effect of renaming target	Continues functioning normally	Link breaks (dangling link)
Effect of deleting target	Preserves data if link count > 0	Link breaks (dangling link)
Can become a dangling link?	No	Yes
Directory linking	Restricted / Not permitted	Supported
Cross-filesystem linking	Not permitted (Invalid cross-device link)	Supported across filesystems (if target filesystem permits)

2. Sorting Techniques (sort)

Suppose alerts.txt contains:

Format: IP-address username failed_attempts

```
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8

$ cat > alerts.txt
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
```

```
$ cat alerts.txt
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
```

Sort according to failed attempts:

```
$ sort -k3,3n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

Sort a list of suspicious IP addresses alphabetically:

```
$ cut -d' ' -f1 alerts.txt | sort
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Display only unique suspicious IP addresses using sort:

```
$ cut -d' ' -f1 alerts.txt | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Sort a list of detected port numbers numerically:

```
$ printf "443\n22\n8080\n80\n21\n" | sort -n
21
22
80
443
8080
```

Display port numbers in descending numerical order:

```
$ printf "443\n22\n8080\n80\n21\n" | sort -nr
8080
443
80
22
21
```

Sort usernames without considering uppercase/lowercase differences:

```
$ cut -d' ' -f2 alerts.txt | sort -f
admin
admin
```

```
guest  
root
```

From a failed-login log, count how many times each IP address occurs and display the most frequent IP first:

```
$ cut -d' ' -f1 alerts.txt | sort | uniq -c | sort -nr  
1 192.168.1.10  
1 172.16.0.5  
1 10.0.0.8  
1 10.0.0.20
```

Given IP, username, failed_attempts, sort records based on the number of failed attempts:

```
$ sort -k3,3n alerts.txt  
172.16.0.5 guest 2  
192.168.1.10 admin 5  
10.0.0.20 admin 8  
10.0.0.8 root 12
```

Sort a CSV vulnerability report based on severity score:

```
$ printf "V001,SQL Injection,9\nV002,XSS,6\nV003,Buffer Overflow,10\nV004,Weak  
Password,3\n" > vulnerability.csv  
  
$ cat vulnerability.csv  
V001,SQL Injection,9  
V002,XSS,6  
V003,Buffer Overflow,10  
V004,Weak Password,3  
  
$ sort -t, -k3,3n vulnerability.csv  
V004,Weak Password,3  
V002,XSS,6  
V001,SQL Injection,9  
V003,Buffer Overflow,10
```

Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses:

```
$ grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' alerts.txt | sort | uniq  
10.0.0.20  
10.0.0.8  
172.16.0.5  
192.168.1.10
```

Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts:

```
$ sort -k3,3nr alerts.txt | head -5 | cut -d' ' -f1  
10.0.0.8  
10.0.0.20  
192.168.1.10  
172.16.0.5
```